

Frontend Requirements & Screen Blueprint

Asset Management System — React Frontend Specification

Objective

Build 5 main React screens/pages covering authentication, dashboard overview, asset inventory, asset allocation, and bills/invoices. The frontend should integrate with the backend APIs and provide a clean, practical UI for Admin, Manager, and Employee workflows.

1. Login Page (/login)

Purpose: User authentication. Admin, Manager, and Employee users will log in according to their assigned role.

Components / Inputs

- Email input field — **type="email"**
- Password input field — **type="password"**
- **Login** submit button

Authentication Flow

- Call backend API: **/api/auth/login**
- Receive JWT token after successful authentication.
- Store the JWT securely using localStorage or cookies, according to the project's security approach.
- Redirect the authenticated user to the dashboard.

2. Dashboard / Home Page (/dashboard)

Purpose: Provide a quick system summary so the user can immediately understand asset quantity and current status.

Stat Cards / Counters

- **Total Assets:** Total number of assets in the system.
- **Available Assets:** Assets currently available for use.
- **Assigned Assets:** Assets currently allotted to users/employees.
- **Under Maintenance:** Assets currently undergoing repair or maintenance.

Quick Actions

- **Add New Asset** shortcut button.
- **Assign Asset** shortcut button.

3. Asset Master / Inventory Page (/assets)

Purpose: Display, search, filter, create, update, and delete physical/digital assets.

Search & Filter Bar

- Search by Asset Name.
- Filter/search by Category.
- Filter by Status: Available / Assigned / Maintenance.

Data Table — Columns

- **Asset Tag:** Barcode / unique asset ID.
- **Asset Name:** Example — Dell Latitude Laptop.
- **Category:** Laptop, Mouse, Chair, etc.

- **Vendor Name.**
- **Purchase Date & Price.**
- **Warranty Expiry Date.**
- **Status Badge:** Green = Available; Yellow = Assigned; Red = Maintenance.
- **Actions:** Edit and Delete buttons.

Add Asset — Modal / Form

- AssetTag
- AssetName
- CategoryID
- VendorName
- PurchaseDate
- Price
- WarrantyExpiryDate

4. Asset Allocation / Assignment Page (/allocations)

Purpose: Track which employee has which asset and support asset assignment and return workflows.

Allocation Table — Columns

- **Asset Name & Tag.**
- **Employee Name:** Employee to whom the asset is allotted.
- **Assigned Date.**
- **Return Date:** Filled when the asset is returned.
- **Remarks / Notes.**

Assign Asset — Modal / Form

- Asset dropdown should show only assets whose status is **Available**.
- User/Employee dropdown.
- AssignedDate.
- Remarks.

Return Asset

- Provide a **Return** action for an assigned asset.
- On successful return, update the asset status back to **Available**.

5. Bills & Invoices Page (/bills)

Purpose: Maintain records of bills/invoices received when assets are purchased and allow users to view or download them.

Bills Table — Columns

- **Bill Number.**
- **Associated Asset Name.**
- **Bill Date.**
- **Amount.**
- **View / Download Bill:** Open PDF or image file.

Upload Bill — Form

- Asset dropdown.
- Bill Number.
- Amount.
- File upload input — **type="file"**.
- Accept PDF and image files.

Recommended Tech Stack / UI Guidance

Use **Tailwind CSS** or **Material UI (MUI)** for styling and ready-made UI components. This is recommended for tables, modals, forms, buttons, badges, dropdowns, and responsive layouts so development time is not wasted on building common UI elements from scratch.

Implementation Checklist

Area	Frontend Requirement
Routing	Implement /login, /dashboard, /assets, /allocations, /bills.
Authentication	Login API integration, JWT handling, protected routes, dashboard redirect.
Reusable UI	Create reusable components for buttons, modals, tables, status badges, forms, and dropdowns.
Asset Status	Available / Assigned / Maintenance states should be visually distinguishable.
Forms	Validate required fields and show clear success/error messages.
Tables	Support search/filtering and action buttons; keep layout responsive.
File Upload	Bills page should support PDF/image uploads and view/download actions.
Responsive Design	Pages should remain usable on desktop and smaller screens.